University of Engineering & Management, Kolkata
Institute of Engineering and Management, Newtown Campus
Department of Computer Applications

ASSIGNMENT SUBMISSION — TOP SHEET

Technical Class Assessment — April 2026
Database Management Systems & Java with JDBC | MCA Programme

STUDENT DETAILS

Full Name	Sohana Singh		
Roll Number	30	**Registration No.**	304202500600090
Section	B	**Semester**	2nd
Programme	MCA	**Academic Year**	2025-26
Email ID	sohanasingh1025@gmail.com		

ASSIGNMENT DETAILS

Subject	Java-DBMS Assignment	**Set Number**	18
Problem Title	The Car Rental System		
Language Used	DBMS & Core JAVA	**Database Used**	MySQL 8.0.44
Submission Date	10th April 2026	**Total Pages**	17

SUBMISSION CHECKLIST

#	Item	Included (/)	Pages / Remarks
1	Printed Java Source Code (DBConnection.java)		
2	Printed Java Source Code (Main.java)		
3	Printed SQL Schema File (schema.sql)		

4 Problem Statement (this top sheet attached)

DECLARATION

I hereby declare that this assignment is my own original work. I have not copied from any other student or used any unauthorised sources. I understand that plagiarism will result in zero marks and disciplinary action.

Student Signature & Date

Faculty Signature & Date

TECHNICAL CLASS ASSESSMENT — April – 2026

Degree: MCA	Stream: MCA
Due Date: _____	**Set No: 18**

The Car Rental

[Domain: Transport / Rental]

Language	Core Java (Console Application)
Database	MySQL
Connectivity	JDBC (Java Database Connectivity)

The Story

> *"Three customers wanted the same car on the same day. I had confirmed all three because I forgot to check."*

A car rental service near Kolkata airport manages a small fleet of vehicles. Bookings are noted on paper, return dates are missed, and the owner never knows which cars are available on a given date without calling each driver.

Your task is to build a Car Rental Booking System using Java and MySQL.

Your Task

Build a Car Rental Booking System as a Java console application using JDBC and MySQL.

Your system must allow:

- Add a car to the fleet (model, registration number, rate per day)
- Book a car for a customer (check availability for the date range)
- Return a car and calculate the final charge based on actual days
- List all cars currently on rent with return due dates

Database Schema

Create the following table(s) in MySQL:

```
CREATE TABLE cars (
    car_id   INT PRIMARY KEY AUTO_INCREMENT,
    model    VARCHAR(100),
    reg_no   VARCHAR(20) UNIQUE,
    rate_day DECIMAL(8,2)
);
```

```
CREATE TABLE rentals
    ( rent_id   INT PRIMARY KEY AUTO_INCREMENT,
    car_id      INT,
    customer    VARCHAR(100),
    from_date   DATE,
    to_date     DATE,
    returned_on DATE DEFAULT NULL,
    FOREIGN KEY (car_id) REFERENCES cars(car_id)
);
```

Required Features

Feature	Implementation Hint
Add Car	INSERT into cars with model, reg_no, rate_day
Book Car	Check no active rental overlaps; INSERT into rentals
Return Car	UPDATE returned_on=TODAY; calculate DATEDIFF(returned_on, from_date) * rate
On Rent List	SELECT cars JOIN rentals WHERE returned_on IS NULL

Constraints

- Use Core Java only — no Spring, Hibernate, or web frameworks
- Connect to MySQL using JDBC (no JPA / ORM)
- All operations must be menu-driven via console
- No GUI required — Scanner-based input is sufficient
- Handle SQL exceptions with try-catch blocks
- Use PreparedStatement for all SQL queries

Sample Interaction

```
===== Car Rental System =====
1. Add Car
2. Book Car
3. Return Car
4. Cars Currently On Rent
5. Exit
Enter choice: 3
Rental ID: 7
Days: 4  |  Rate: Rs 2000/day  |  Total: Rs 8000.00
```

Submission Checklist

- DBConnection.java (JDBC connection class)
- Main.java (menu-driven main program)
- SQL file: schema.sql (CREATE TABLE statements)
- screenshot.png (terminal output showing at least 2 features working)

> *"A database is not just data. It is the memory of your application. Make it reliable."*

SRC/MAIN.JAVA

```
import java.util.Scanner;
public class Main {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        CarService service = new CarService();
        while (true) {

            // ---- Print the menu ----
            System.out.println("\n===== Car Rental System =====");
            System.out.println("1. Add Car");
            System.out.println("2. Book Car");
            System.out.println("3. Return Car");
            System.out.println("4. Cars Currently On Rent");
            System.out.println("5. Exit");
            System.out.print("Enter choice: ");
            int choice = sc.nextInt();
            sc.nextLine();
            switch (choice) {

                case 1: // ADD CAR
                    System.out.print("Enter car model: ");
                    String model = sc.nextLine().trim();  // .trim() removes
accidental spaces

                    System.out.print("Enter registration number: ");
                    String regNo = sc.nextLine().trim();

                    System.out.print("Enter rate per day (Rs): ");
                    double rate = sc.nextDouble();
                    sc.nextLine();  // consume newline after double input

                    service.addCar(model, regNo, rate);
                    break;
                case 2: // BOOK CAR
                    System.out.print("Enter Car ID to book: ");
                    int carId = sc.nextInt();
                    sc.nextLine();  // consume newline

                    System.out.print("Enter customer name: ");
                    String customer = sc.nextLine().trim();
```

```
                // Dates must be in YYYY-MM-DD format for MySQL DATE
columns
                System.out.print("Enter from date (YYYY-MM-DD): ");
                String fromDate = sc.nextLine().trim();

                System.out.print("Enter to date (YYYY-MM-DD): ");
                String toDate = sc.nextLine().trim();

                service.bookCar(carId, customer, fromDate, toDate);
                break;
            case 3: // RETURN CAR
                System.out.print("Enter Rental ID: ");
                int rentId = sc.nextInt();
                sc.nextLine();  // consume newline

                service.returnCar(rentId);
                break;

            case 4: // LIST CARS ON RENT
                service.listCarsOnRent();
                break;
            case 5: // EXIT
                System.out.println("Exiting... Thank you!");
                sc.close();     // close Scanner to release resources
                System.exit(0); // terminate JVM with success code 0
                break;
            default: // INVALID CHOICE
                System.out.println("Invalid choice. Please enter 1–5.");
        }
    }
  }
}
```

SRC/DBCONNECTION.JAVA

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
public class DBConnection {

    private static final String URL      =
"jdbc:mysql://localhost:3306/car_rental_db";
    private static final String USER     = "root";     // change if your MySQL
user is different
    private static final String PASSWORD = "Alpha_@123";     // change to your
MySQL password

    /**
     * getConnection()
     * ------------------
     * Static method – can be called without creating an object.
     * Loads the MySQL JDBC driver and returns a live Connection object.
     *
     * @return  Connection  – active database connection
     * @throws  SQLException if connection fails (wrong URL/credentials/DB not
running)
     */
    public static Connection getConnection() throws SQLException {
        try {
            // Explicitly load the MySQL JDBC driver class into the JVM.
            // Required for older JDBC versions; harmless in newer ones.
            Class.forName("com.mysql.cj.jdbc.Driver");
        } catch (ClassNotFoundException e) {
            // Driver JAR not on classpath → show helpful message
            System.out.println("MySQL Driver not found. Add mysql-connector-
java.jar to classpath.");
            e.printStackTrace();
        }

        // DriverManager uses the URL + credentials to create a TCP connection
        // to MySQL and returns a Connection object we can use to run SQL.
        return DriverManager.getConnection(URL, USER, PASSWORD);
    }
}
```

SRC/CARSERVICE.JAVA

```
import java.sql.*;
import java.time.LocalDate;

public class CarService {

    //
==================================================================
==
    // FEATURE 1: ADD CAR
    //
==================================================================
==

    /**
     * addCar()
     * ----------
     * Inserts a new car record into the `cars` table.
     *
     * @param model   – car model name  e.g. "Maruti Swift"
     * @param regNo   – registration number  e.g. "WB01AB1234"
     * @param rateDay – rental rate per day in Rs  e.g. 1500.00
     */
    public void addCar(String model, String regNo, double rateDay) {
        // SQL with ? placeholders → PreparedStatement fills them safely
        // PreparedStatement prevents SQL Injection attacks
        String sql = "INSERT INTO cars (model, reg_no, rate_day) VALUES
(?, ?, ?)";

        // try-with-resources → automatically closes Connection &
PreparedStatement
        // even if an exception occurs (no need for finally block)
        try (Connection con = DBConnection.getConnection();
             PreparedStatement ps = con.prepareStatement(sql)) {

            // Bind values to the ? placeholders (index starts at 1, not 0)
            ps.setString(1, model);     // 1st ?  = model
            ps.setString(2, regNo);     // 2nd ?  = reg_no
            ps.setDouble(3, rateDay);  // 3rd ?  = rate_day

            // executeUpdate() runs INSERT/UPDATE/DELETE; returns rows
affected
            int rows = ps.executeUpdate();
```

```
            if (rows > 0) {
                System.out.println("Car added successfully!");
            } else {
                System.out.println("Failed to add car.");
            }

        } catch (SQLException e) {
            // SQLIntegrityConstraintViolationException is a subclass of
SQLException
            // It fires if reg_no is not UNIQUE (duplicate registration number)
            System.out.println("Error adding car: " + e.getMessage());
        }
    }

    //
==================================================================
==
    // FEATURE 2: BOOK CAR
    //
==================================================================
==

    /**
     * bookCar()
     * -----------
     * Books a car for a customer for a given date range.
     * Before booking, checks if the car is available (no overlapping rentals).
     *
     * Overlap logic:
     *   An overlap exists when:
     *   existing.from_date <= requested.to_date
     *   AND
     *   existing.to_date >= requested.from_date
     *
     * @param carId      – ID of the car to book
     * @param customer   – customer's name
     * @param fromDate   – start date (YYYY-MM-DD)
     * @param toDate     – end date   (YYYY-MM-DD)
     */
    public void bookCar(int carId, String customer, String fromDate, String toDate)
{

        // ---- Step 1: Check if the requested car exists ----
```

```
        String checkCarSql = "SELECT car_id FROM cars WHERE car_id = ?";

        // ---- Step 2: Check for overlapping active rentals ----
        // A rental is "active" if returned_on IS NULL (car not yet returned)
        // We look for any rental of this car where dates overlap with our request
        String overlapSql =
            "SELECT COUNT(*) FROM rentals " +
            "WHERE car_id = ? " +
            "AND returned_on IS NULL " +
            "AND from_date <= ? " +        // existing rental starts before our
end date
            "AND to_date   >= ?";          // existing rental ends after our start
date

        // ---- Step 3: Insert the new rental ----
        String insertSql =
            "INSERT INTO rentals (car_id, customer, from_date, to_date) " +
            "VALUES (?, ?, ?, ?)";

        try (Connection con = DBConnection.getConnection()) {

            // --- Check car exists ---
            try (PreparedStatement ps = con.prepareStatement(checkCarSql)) {
                ps.setInt(1, carId);
                ResultSet rs = ps.executeQuery();
                if (!rs.next()) {
                    System.out.println("Car ID " + carId + " not found.");
                    return;  // exit method early
                }
            }

            // --- Check overlap ---
            try (PreparedStatement ps = con.prepareStatement(overlapSql)) {
                ps.setInt(1, carId);
                ps.setString(2, toDate);    // our toDate must be < existing
fromDate
                ps.setString(3, fromDate);  // our fromDate must be > existing
toDate
                ResultSet rs = ps.executeQuery();
                rs.next();  // COUNT(*) always returns exactly one row
                int count = rs.getInt(1);  // get the count value

                if (count > 0) {
                    // Car is already booked during the requested period
```

```
                System.out.println("Car is NOT available for the selected
dates.");
                return;  // exit method early, don't book
            }
        }

        // --- Insert rental ---
        try (PreparedStatement ps = con.prepareStatement(insertSql)) {
            ps.setInt(1, carId);
            ps.setString(2,  customer);
            ps.setString(3, fromDate);
            ps.setString(4, toDate);
            int rows = ps.executeUpdate();
            if (rows > 0) {
                System.out.println("Car booked successfully for " + customer +
"!");
            }
        }

    } catch (SQLException e) {
        System.out.println("Error booking car: " + e.getMessage());
    }
}

//
==================================================================
==
// FEATURE 3: RETURN CAR
//
==================================================================
==

/**
 * returnCar()
 * --------------
 * Marks a rental as returned (sets returned_on = TODAY),
 * then calculates and displays the total charge.
 *
 * Charge formula:
 *   days  = DATEDIFF(returned_on, from_date)   [MySQL function]
 *   total = days * rate_day
 *
 * @param rentId – the ID of the rental record to close
 */
```

```
    public void returnCar(int rentId) {

        // Get today's date from Java (alternatively could use MySQL's
CURDATE())
        String today = LocalDate.now().toString();   // format: "YYYY-MM-DD"

        // ---- Step 1: Update returned_on to today ----
        String updateSql =
            "UPDATE rentals SET returned_on = ? WHERE rent_id = ? AND
returned_on IS NULL";
        //  ^--- Only update if returned_on is still NULL (not already returned)

        // ---- Step 2: Fetch rental + car rate to calculate charge ----
        String fetchSql =
            "SELECT r.from_date, r.returned_on, c.rate_day, r.customer " +
            "FROM rentals r " +
            "JOIN cars c ON r.car_id = c.car_id " +  // JOIN to get rate_day
from cars table
            "WHERE r.rent_id = ?";

        try (Connection con = DBConnection.getConnection()) {

            // --- Update the return date ---
            try (PreparedStatement ps = con.prepareStatement(updateSql)) {
                ps.setString(1, today);
                ps.setInt(2, rentId);
                int rows = ps.executeUpdate();

                if (rows == 0) {
                    // Either rentId doesn't exist OR car already returned
                    System.out.println("Rental ID not found or already returned.");
                    return;
                }
            }

            try (PreparedStatement ps = con.prepareStatement(fetchSql)) {
                ps.setInt(1, rentId);
                ResultSet rs = ps.executeQuery();

                if (rs.next()) {
                    String fromDate     = rs.getString("from_date");
                    String returnedOn   = rs.getString("returned_on");
                    double ratePerDay   = rs.getDouble("rate_day");
                    String customerName = rs.getString("customer");
```

```
                LocalDate start  = LocalDate.parse(fromDate);
                LocalDate end    = LocalDate.parse(returnedOn);
                long days        = end.toEpochDay() - start.toEpochDay();  //
difference in days
                if (days <= 0) days = 1;

                double totalCharge = days * ratePerDay;

                // Print the receipt
                System.out.println("\n===== Return Receipt =====");
                System.out.println("Customer    : " + customerName);
                System.out.println("Rental ID  : " + rentId);
                System.out.println("From        : " + fromDate);
                System.out.println("Returned    : " + returnedOn);
                System.out.printf ("Days        : %d%n", days);
                System.out.printf ("Rate        : Rs %.2f/day%n", ratePerDay);
                System.out.printf ("Total       : Rs %.2f%n", totalCharge);
                System.out.println("==========================\n");
            }
        }

    } catch (SQLException e) {
        System.out.println("Error returning car: " + e.getMessage());
    }
}
public void listCarsOnRent() {

    String sql =
        "SELECT r.rent_id, c.model, c.reg_no, r.customer, " +
        "       r.from_date, r.to_date " +
        "FROM rentals r " +
        "JOIN cars c ON r.car_id = c.car_id " +
        "WHERE r.returned_on IS NULL " +
        "ORDER BY r.to_date ASC";

    try (Connection con = DBConnection.getConnection();
         PreparedStatement ps = con.prepareStatement(sql);
         ResultSet rs = ps.executeQuery()) {  // executeQuery() for SELECT
statements

        System.out.println("\n====== Cars Currently On Rent ======");
        System.out.printf("%-10s %-20s %-15s %-20s %-12s %-12s%n",
                          "RentID", "Model", "Reg No", "Customer", "From", "Due
Back");
```

```
            System.out.println("-----------------------------------------------------
-----------------------------------------------------");

            boolean found = false;

            // rs.next() moves the cursor to the next row; returns false when no
more rows
            while (rs.next()) {
                found = true;
                int     rentId    = rs.getInt("rent_id");
                String model     = rs.getString("model");
                String regNo     = rs.getString("reg_no");
                String customer = rs.getString("customer");
                String fromDate = rs.getString("from_date");
                String toDate    = rs.getString("to_date");

                // printf with %-Ns = left-aligned string in N-char wide column
                System.out.printf("%-10d %-20s %-15s %-20s %-12s %-12s%n",
                                  rentId, model, regNo, customer, fromDate, toDate);
            }

            if (!found) {
                System.out.println("No cars are currently on rent.");
            }

            System.out.println("=====================================\n");

        } catch (SQLException e) {
            System.out.println("Error fetching rentals: " + e.getMessage());
        }
    }
}
```

SQL/SCHEMA.SQL

```
CREATE DATABASE IF NOT EXISTS car_rental_db;
USE car_rental_db;

-- Table 1: cars
-- Stores the fleet of vehicles available for rent
CREATE TABLE cars (
    car_id     INT PRIMARY KEY AUTO_INCREMENT,    -- unique ID for each car
    model      VARCHAR(100),                      -- car model name e.g. "Swift
Dzire"
    reg_no     VARCHAR(20) UNIQUE,                -- registration number, must
be unique
    rate_day   DECIMAL(8,2)                       -- rental rate per day in Rs.
);


-- Table 2: rentals
-- Stores all booking/rental records
CREATE TABLE rentals (
    rent_id      INT PRIMARY KEY AUTO_INCREMENT, -- unique ID for each
rental
    car_id       INT,                            -- which car is rented (FK to cars)
    customer     VARCHAR(100),                   -- customer name
    from_date    DATE,                           -- rental start date
    to_date      DATE,                           -- expected return date
    returned_on DATE DEFAULT NULL,               -- actual return date (NULL =
still on rent)
    FOREIGN KEY (car_id) REFERENCES cars(car_id)
);
```

Screensorts

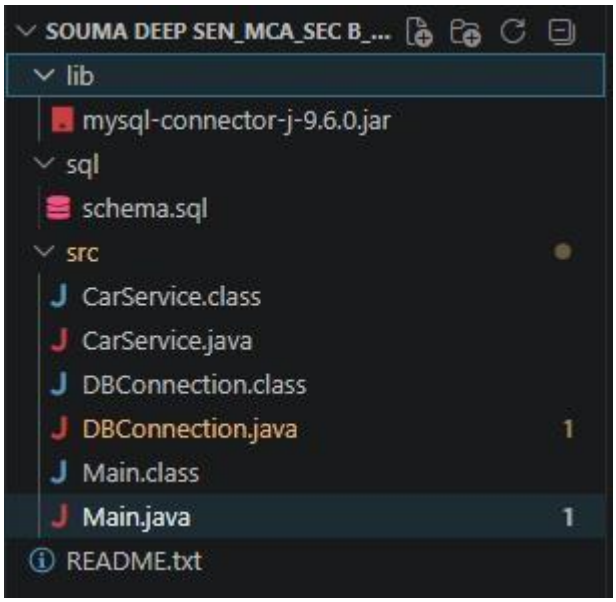
```
===== Car Rental System =====
1. Add Car
2. Book Car
3. Return Car
4. Cars Currently On Rent
5. Exit
Enter choice: 1
Enter car model: Tata Nexon
Enter registration number: WB04GH3456
Enter rate per day (Rs): 1800
Car added successfully!

===== Car Rental System =====
1. Add Car
2. Book Car
3. Return Car
4. Cars Currently On Rent
5. Exit
Enter choice: 2
Enter Car ID to book: 2
Enter customer name: Amit Singh
Enter from date (YYYY-MM-DD): 2026-03-28
Enter to date (YYYY-MM-DD): 2026-04-02
Car booked successfully for Amit Singh!

===== Car Rental System =====
1. Add Car
2. Book Car
3. Return Car
4. Cars Currently On Rent
5. Exit
Enter choice: 3
Enter Rental ID: 2

===== Return Receipt =====
Customer   : Amit Singh
Rental ID  : 2
From       : 2026-03-28
Returned   : 2026-04-03
Days       : 6
Rate       : Rs 2000.00/day
Total      : Rs 12000.00
==========================
```

```
===== Car Rental System =====
1. Add Car
2. Book Car
3. Return Car
4. Cars Currently On Rent
5. Exit
Enter choice: 4

====== Cars Currently On Rent ======
RentID     Model                Reg No          Customer            From        Due Back
-------------------------------------------------------------------------------------
No cars are currently on rent.
=====================================
```

FOLDER STRUCTURE